

Transport Layer

TCP • Congestion Control • Flow Control • Timers

Week 3 — Networks & Systems

TCP Fundamentals & Segment Format

Flow & Error Control

TCP Connection Management

Sliding Windows & ACK Rules

TCP Timers & RTO Calculation

Congestion Control (AIMD)

TCP – Transmission Control Protocol

Foundation of reliable, ordered, connection-oriented communication

Connection-Oriented



TCP establishes a virtual connection (three-way handshake) before any data is sent. This guarantees both ends are ready.

Full-Duplex



Data flows simultaneously in both directions within a single TCP connection — sender and receiver roles exist on both sides.

Flow & Error Control



TCP uses receive windows (rwnd) to pace data and sequence numbers / ACKs to detect and retransmit lost or corrupted segments.



Key: TCP is byte-stream, not message-oriented. Data is divided into segments and each byte is numbered for reliable delivery.

Process-to-Process Delivery: Well-Known Ports

TCP uses port numbers to identify the application-layer process

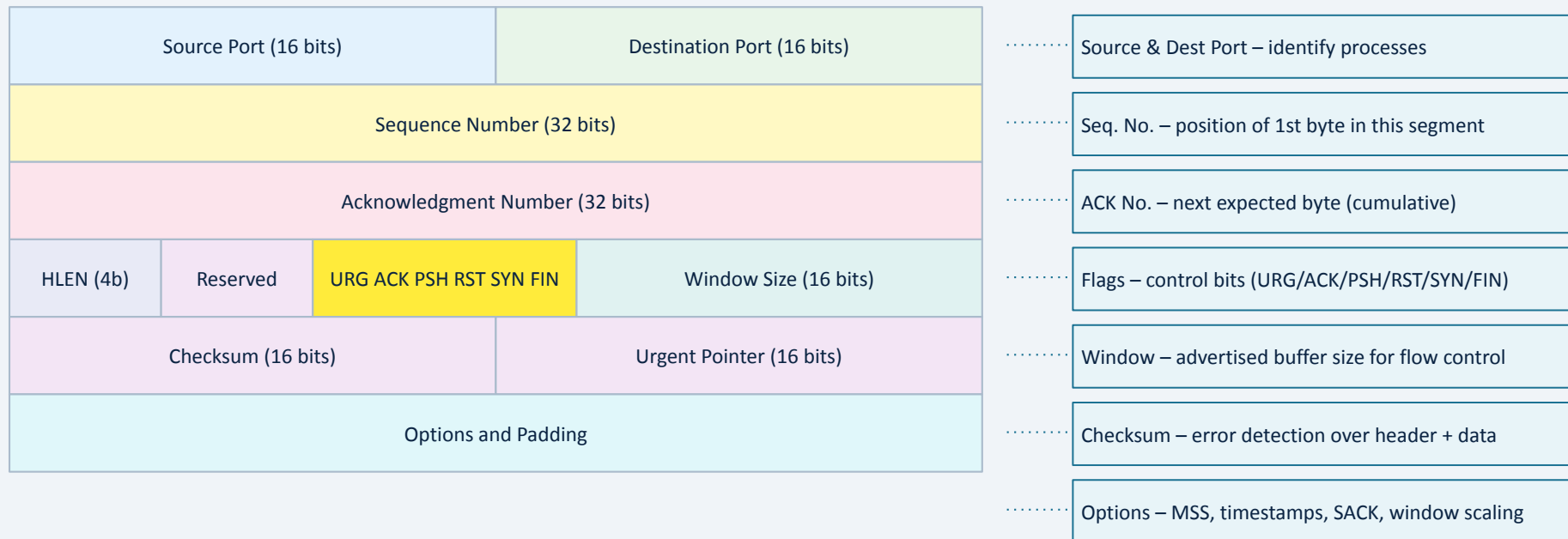
Port	Protocol	Description
7	Echo	Echoes a received datagram back to the sender
20/21	FTP	File Transfer Protocol (data & control connections)
23	TELNET	Terminal Network – remote login
25	SMTP	Simple Mail Transfer Protocol
53	DNS	Domain Name Server
80	HTTP	Hypertext Transfer Protocol (web)
111	RPC	Remote Procedure Call



Ports 0–1023 are reserved 'well-known' ports assigned by IANA. Ports 1024–49151 are registered; 49152–65535 are dynamic/ephemeral.

TCP Segment Format

Every TCP segment carries a header (≥ 20 bytes) followed by data



TCP Control Flags (6-Bit Field)

Each bit has a distinct purpose in managing connection state

URG

Urgent Pointer is valid. Receiver delivers urgent data immediately without buffering. Used for out-of-band signaling.

ACK

Acknowledgment field is valid. Present in all segments after connection establishment. Confirms receipt of data.

PSH

Push function. Tells receiver to flush buffered data to the application immediately instead of waiting for buffer to fill.

RST

Reset the connection. Sent when a TCP segment arrives for a port with no listener, or to abort a connection immediately.

SYN

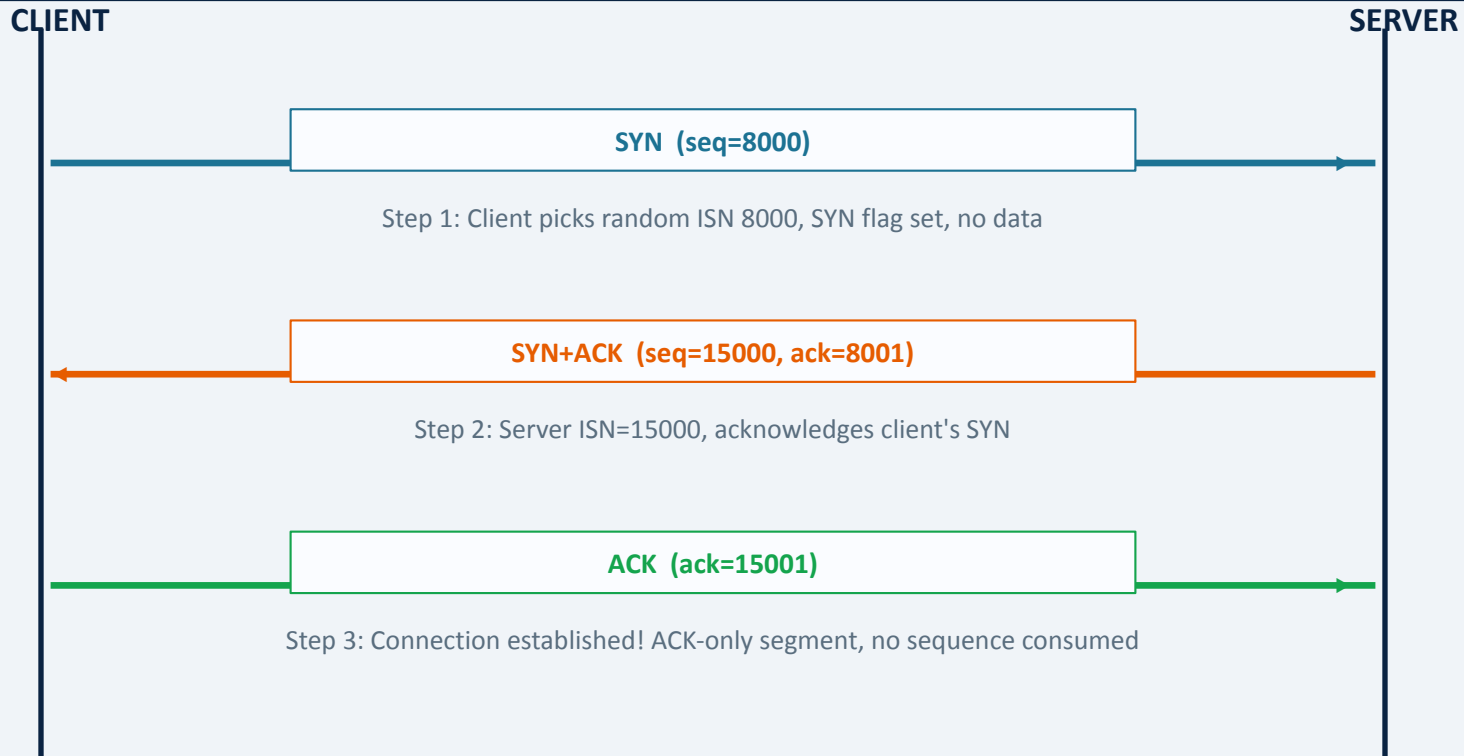
Synchronize sequence numbers. Used **ONLY** during connection establishment (SYN, SYN+ACK). Consumes one sequence number.

FIN

Finish / terminate. Used to close a connection gracefully. Also consumes one sequence number if no data is carried.

TCP Connection Establishment: Three-Way Handshake

Client initiates (active open), server listens (passive open)



SYN floods: attackers send many SYNs with spoofed IPs, filling the server's half-open queue. Mitigation: SYN cookies.

Sequence & Acknowledgment Numbers

Every byte of data transferred in a TCP connection is numbered

Byte-Level Numbering

Each byte of data has a unique sequence number. The sequence number in a segment = the number of its FIRST byte.

Random ISN

The Initial Sequence Number (ISN) is randomly generated to prevent old connection interference and improve security.

Cumulative ACK

The acknowledgment number = next byte expected. Receiving ACK=5001 means all bytes up to 5000 were received.

Example: 5 Segments of 1000 bytes each, ISN = 10000

Segment	Seq Number	Byte Range	ACK Expected
1	10,001	10,001 – 11,000	11,001
2	11,001	11,001 – 12,000	12,001
3	12,001	12,001 – 13,000	13,001
4	13,001	13,001 – 14,000	14,001
5	14,001	14,001 – 15,000	15,001

Flow Control & Sliding Windows

Prevents the sender from overwhelming the receiver's buffer

How It Works

- ① Receiver advertises `rwnd` (receive window) in every ACK
- ② Sender keeps outstanding bytes $\leq$ `rwnd` at all times
- ③ When receiver consumes data, `rwnd` grows $\rightarrow$ window opens
- ④ If receiver buffer fills, `rwnd` = 0 $\rightarrow$ sender pauses (zero window)
- ⑤ Persistence timer fires probe segments to re-check `rwnd`

Window Size Formulas

`rwnd` = Buffer Size – Unprocessed Bytes

e.g. $5000 - 1000 = 4000$ bytes

`cwnd` = Congestion window (set by sender)

Starts at 1 MSS, grows via slow-start/AIMD

Effective Window = $\min(\text{rwnd}, \text{cwnd})$

e.g. $\min(3000, 3500) = 3000$ bytes

Deadlock risk: if `rwnd`=0 ACK is lost, sender waits forever. Solution: Persistence Timer sends periodic probes.

Rules for Generating ACKs

TCP uses delayed ACK and duplicate ACK strategies

Piggybacking

- 1 When one end sends data, it piggybacks an ACK for the data received so far, saving a separate ACK packet.

Delayed ACK (500 ms)

- 2 If only one in-order segment is outstanding, wait up to 500 ms for another segment or data to send — then ACK. Reduces traffic.

Max 2 Unacked Segments

- 3 Never allow more than 2 consecutive un-ACKed in-order segments. Prevents unnecessary retransmission from sender.

Immediate Duplicate ACK

- 4 If an out-of-order segment arrives (gap detected), immediately send a duplicate ACK for the last in-order byte. Triggers fast retransmit.

Fill-the-Gap ACK

- 5 When the missing segment finally arrives, send an ACK to announce the next expected sequence number.

Duplicate Segment ACK

- 6 If a segment already received arrives again (duplicate), immediately send an ACK — so sender stops retransmitting.

TCP Timers

Most TCP implementations use at least four timers for reliable operation



Retransmission (RTO)

Started when a segment is sent; stopped when ACK arrives. If timer expires, the segment is retransmitted. RTO is dynamically calculated from measured RTT.



Persistence

Handles zero-window deadlock. When $rwnd=0$, persistence timer fires and sends a probe segment. Doubles (like RTO backoff) up to 60 s until window reopens.



Keepalive

Prevents long-idle connections. After 2 hours of inactivity, TCP sends 10 probe segments at 75-second intervals. If no response, connection is closed.



TIME-WAIT (2MSL)

After sending last ACK in connection close, the closing side waits $2 \times$ MSL (Maximum Segment Lifetime, typically 2 min) before fully closing. Prevents old segments from interfering with new connections.

RTO Calculation – Jacobson's Algorithm

TCP adapts its retransmission timeout dynamically using smoothed RTT

Smoothed RTT (RTTS)

$$RTTS = (7/8) \times RTTS + (1/8) \times RTTM$$

Exponential weighted moving average ($\alpha = 1/8$)

RTT Deviation (RTTD)

$$RTTD = (3/4) \times RTTD + (1/4) \times |RTTS - RTTM|$$

Measures variance of RTT ($\beta = 1/4$)

Retransmission Timeout (RTO)

$$RTO = RTTS + 4 \times RTTD$$

4 σ rule — covers most real-world RTT jitter

Worked Example: $RTTM=1.5$ s (1st measure) $\rightarrow$ $RTTS=1.5$ $RTTD=0.75$ $RTO=1.5+4 \times 0.75=4.5$ s

Karn's Algorithm: Never use RTT of a retransmitted segment

Exponential Backoff: RTO doubles on each retransmission

Congestion – What It Is & Why It Matters

Network load exceeds capacity → packets delayed or dropped

Congestion occurs when the number of packets sent to the network is greater than the capacity of the network to handle them.



Delay vs Load

Delay remains low while load is below capacity. Once load exceeds capacity (congestion zone), queues fill and delay spikes exponentially. Packets time out and are retransmitted, worsening congestion.



Throughput vs Load

Throughput increases with load up to the capacity point. Beyond that, congestion causes widespread packet drops and retransmissions. Actual throughput FALLS even as more packets are injected.

Congestion collapse: extreme congestion → throughput approaches zero. First observed in ARPANET 1986.

Congestion Control: Open-Loop vs Closed-Loop

Two broad categories of congestion management mechanisms

OPEN-LOOP (Prevention)

Design the system so congestion never happens

- ▶ Retransmission Policy – when to retransmit
- ▶ Window Policy – size & type of window
- ▶ Acknowledgment Policy – when to ACK
- ▶ Discarding Policy – which packets to drop first
- ▶ Admission Policy – refuse new connections if congested

CLOSED-LOOP (Removal)

Monitor, detect, inform, adjust — feedback loop

Back Pressure

Downstream router signals upstream to slow sending (VC only)

Choke Packet

Congested router sends ICMP Source Quench directly to sender

Implicit Signaling

Sender infers congestion from timeout or duplicate ACKs (TCP)

Explicit Signaling

BECN/FECN bits in Frame Relay; ECN bits in IP header

TCP Congestion Control – AIMD Algorithm

Additive Increase / Multiplicative Decrease with Slow Start



Slow Start

cwnd starts at 1 MSS. After each ACK received, $\text{cwnd} += 1 \text{ MSS}$ → doubles each RTT (exponential growth). Continues until $\text{cwnd} \geq \text{ssthresh}$.



Congestion Avoidance

Once $\text{cwnd} \geq \text{ssthresh}$, increase cwnd by 1 MSS per RTT (additive). Probes carefully for available bandwidth without risking collapse.



Timeout Detected

$\text{ssthresh} = \text{cwnd}/2$ | $\text{cwnd} = 1 \text{ MSS}$. Reset to slow start. Severe congestion signal — connection must restart conservatively.

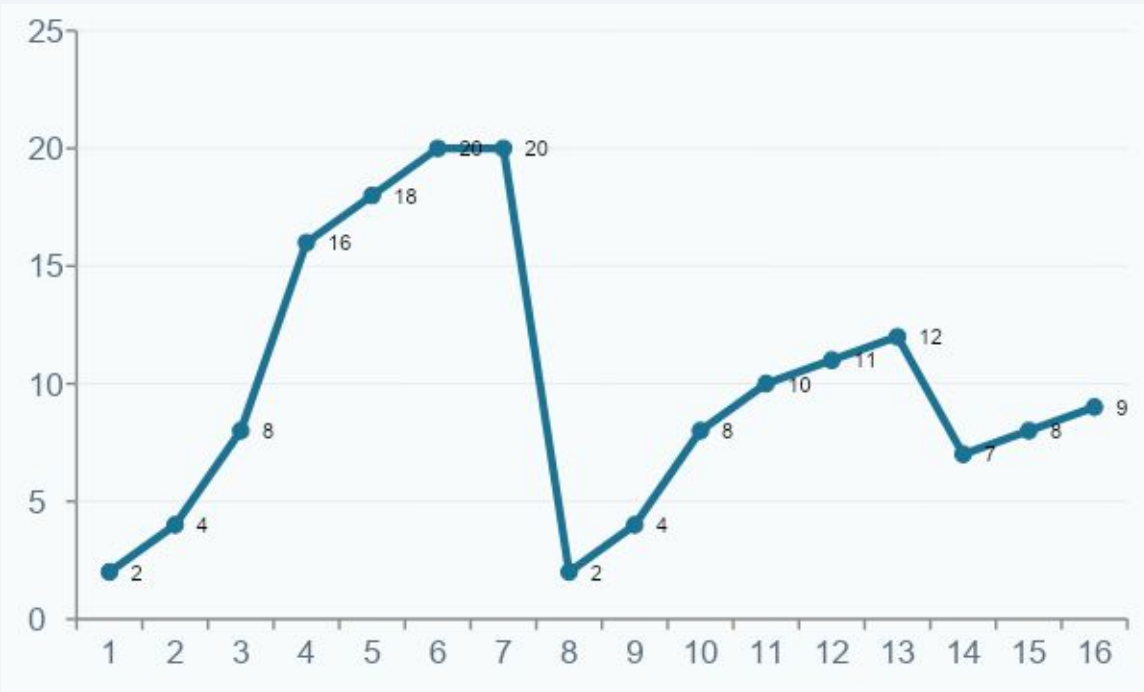


3 Dup ACKs Detected

$\text{ssthresh} = \text{cwnd}/2$ | $\text{cwnd} = \text{ssthresh}$. Jump to congestion avoidance (Fast Recovery). Mild congestion — no need for full slow-start reset.

Congestion Control – Worked Example

Tracing cwnd through Slow Start, Congestion Avoidance, and Multiplicative Decrease



Rnd 1–4 — Slow Start (SS)

cwnd doubles: 2→4→8→16. Threshold=16.

Rnd 5–7 — Congestion Avoidance (AI)

cwnd: 16+1=17, +1=18... until timeout at 20.

Rnd 8 — Timeout → MD

ssthresh=10, cwnd=1. New slow start.

Rnd 9–11 — Slow Start again

cwnd doubles: 2→4→8→10 (hits new ssthresh).

Rnd 12–13 — Congestion Avoidance

AI: 10→11→12. 3 dup ACKs at 12.

Key: Timeout = more severe (restart from 1 MSS). 3-Dup-ACKs = milder (Fast Recovery preserves higher cwnd).

Traffic Shaping: Leaky Bucket & Token Bucket

Open-loop techniques to smooth bursty traffic before it enters the network



Leaky Bucket

- ▶ Incoming packets enter a fixed-size queue (bucket)
- ▶ Packets leave at a CONSTANT rate regardless of arrival rate
- ▶ If bucket is full → excess packets are DISCARDED
- ▶ Converts bursty traffic into smooth, fixed-rate output
- ▶ Disadvantage: always outputs at fixed rate even when network is free
- ▶ Example: Input 12 Mbps burst → Output 3 Mbps steady stream



Token Bucket

- ▶ Tokens added to bucket at a fixed rate (1 per tick)
- ▶ A packet can only be sent if a token is available
- ▶ Token consumed for each cell/packet transmitted
- ▶ Bucket has maximum capacity — excess tokens discarded
- ▶ Allows BURSTS as long as tokens are accumulated
- ▶ Advantage: permits short bursts while enforcing average rate

Week 3 – Transport Layer: Key Takeaways



TCP is connection-oriented, full-duplex, and byte-stream based. Setup via 3-way handshake (SYN/SYN-ACK/ACK).



Segments carry 32-bit seq/ack numbers. Seq = first byte in segment. ACK = next byte expected (cumulative).



Flow Control uses rwnd to prevent buffer overflow. Effective window = $\min(\text{rwnd}, \text{cwnd})$.



$\text{RTO} = \text{RTTS} + 4 \times \text{RTTD}$, updated with Jacobson's algo. Karn's algo excludes retransmitted RTT. Backoff on retry.



Congestion Control: Slow Start (exponential) → Congestion Avoidance (additive) → Multiplicative Decrease on congestion.



Traffic shaping: Leaky Bucket = fixed output rate (drops excess); Token Bucket = allows bursts up to bucket capacity.